

Your Neighbours, Not Your Autoscaler: Protecting Reclaimable Prefixes in Agentic LLM Serving

Anonymous Author(s)
Submission #xxx

Abstract

Agent programs alternate model invocations with tool calls. During a tool call, GPU utilisation, request queues, and exported active-KV utilisation can all be zero even though a long prefix may make the next turn cheap. We initially assumed that native vLLM reserved this state and that scale-in was its main threat. Hardware validation falsified both premises: completed requests leave reclaimable, metric-invisible prefix content, and neighboring requests can evict it before an autoscaler acts.

On Qwen2.5-14B/vLLM 0.25.1, losing a 32k prefix adds 9.03 s. A parked 16k prefix remains intact behind at most eight concurrent 4k contexts, is 64.5% retained behind 16, and is gone behind 24, while 0–32 s minimum parking time has little independent effect. A second hardware test submits the same 24 neighbors either together or as eight before the probe plus sixteen after it. Admission protection changes survival from 0% to 100% and probe TTFT from 3.735 s to 0.124 s without dropping any work.

We correct a trace-driven simulator, include dispatch queueing in the SLO, and evaluate eight dynamic baselines at 8 and 64 turns. Across four controllers and four turn-count regimes, 96.53% of destroyed prefix tokens are due to pressure eviction and 3.47% to scale-in. A P1-derived pressure-aware admission baseline eliminates simulated pressure eviction. At 8 turns and $8\times$ synthetic load it attains 1.000 SLO versus HPA’s 0.997 while using 45.6% fewer GPU-seconds. At 64 turns and $8\times$, it attains 0.991 versus 0.979 and saves 326M recomputed tokens, but uses 8.2% more GPU. Admission converts destructive recomputation into bounded queueing; whether it saves GPU depends on trajectory length. The useful control variable is pressure on valuable reclaimable prefixes, not idle time alone.

Keywords

LLM serving, agents, autoscaling, prefix cache, admission control

1 Introduction

An LLM agent is a program rather than a request. It invokes a model, runs an external tool, appends the result, and invokes the model again on an extended transcript [2]. Between invocations it uses no GPU. We call this interval *parked*. The accumulated prefix can still be valuable: losing it forces a re-prefill before the next turn.

This creates a tempting but incorrect mental model. If a serving engine pins a parked program’s KV blocks, compute-side signals miss the program while a memory-side signal sees it. Scaling in then destroys hard-held state. Our original

study encoded exactly that model and produced a strong result for a controller that counted parked programs.

We designed a four-part vLLM validation to measure the assumptions before using them in the paper. The experiment did what validation should do: it falsified the central one. Native vLLM returns a completed request’s blocks to its free/reclaimable pool. Prefix caching may preserve their content, but the program has no reservation and vLLM’s exported active-KV metric reports no allocation for it. During a tool call all three deployed signals can therefore read zero simultaneously:

- GPU utilisation sees no kernel executing for the program;
- queue/running metrics see no submitted request; and
- active-KV utilisation sees no blocks allocated to an active request.

The state is neither resident in the resource-accounting sense nor absent in the performance sense. It is an opportunistic, evictable option on a future cache hit. This distinction changes both the model and the control problem.

This paper makes three contributions. First, it measures the missing state, its price, and its survival under time and pressure on a real vLLM instance. Second, it corrects the simulator to separate active allocation from reclaimable prefix content, includes queueing in end-to-end slowdown, and shows that pressure eviction accounts for 96.53% of baseline prefix destruction. Third, it reports that the original PARKAWARE mechanism no longer wins reliably and evaluates the data-directed alternative: couple admission and scale-out to cache pressure. Hardware P2, 72 E8 policy points, a 28-point admission sweep, and explicit queue/recompute/GPU accounting test the resulting mechanism.

2 Workload and Signals

2.1 Agentic serving

Context accumulates across an agent trajectory. Turn i contains prior model outputs and tool results, so a later prompt may be tens of thousands of tokens. Serving systems use paged KV management and prefix caching to avoid repeating that prefill [3]. Published coding-agent runs contain long trajectories; we therefore sweep up to 64 turns rather than treating every invocation as an independent chat request [10].

Microsoft’s public inference dataset provides one conversation trace and one coding trace [1]. The coding trace contains 8,819 requests with a context-to-generation ratio of 73:1, index of dispersion 71.3, and peak-to-mean arrival rate 12.8. The conversation trace contains 19,366 requests, with

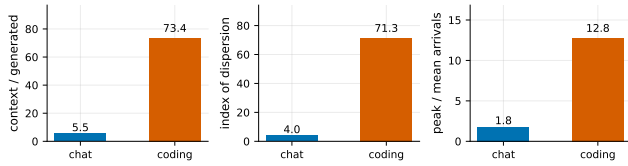


Figure 1: Measured properties of the two Azure traces.

the corresponding values 5.5:1, 4.0, and 1.8 (Figure 1). The coding trace is not a trace of tool calls, but its long-context, short-output and bursty shape makes it a useful request-level substrate.

2.2 Three zeroes, one latent value

GPU utilisation, queue depth, and active-KV utilisation are sensible signals for submitted requests. A parked program is outside that interface. Yet a cache hit can make its next turn much cheaper. We denote a program’s cached prefix by c_i and the fraction still present when it returns by $q_i \in [0, 1]$. Its latent recompute liability is

$$L_i = \frac{(1 - q_i)c_i}{R_{\text{prefill}}}. \quad (1)$$

Neither q_i nor the program’s probability of returning is exported by the three native autoscaling signals. Counting programs alone supplies neither.

3 Hardware Validation

3.1 Setup

We ran vLLM 0.25.1 with prefix caching enabled, using the Qwen2.5-14B Instruct checkpoint on one device reported as RTX 4090 with 49,140 MiB. The server used a 32,768-token maximum sequence length and reported capacity for 75,216 KV-cache tokens. Each point has three repetitions unless noted. The repository contains the exact scripts, raw CSVs, server logs, and a machine-readable summary.

3.2 V1: price of a prefix

We issue the same prefix twice for a hit and a distinct prefix for a miss. At 1k tokens, the mean miss-hit penalty is 0.161 s; at 32k it is 9.028 s and the hit is 44.2× faster. A linear fit gives 0.286 s per 1k tokens with $R^2 = 0.989$. The state is valuable even though it is not reserved.

3.3 V2: the assumption that failed

Eight sessions alternate model calls with 8 s tool sleeps while DCGM and the vLLM Prometheus endpoint are sampled. Across 114 samples in which all sessions are parked, mean GPU utilisation, running requests, waiting requests, and active-KV utilisation are all exactly zero. When any session computes, the corresponding means are 93.5% GPU, 5.96 running, 0.62 waiting, and 29.5% active KV. Thus KV-util does not rescue observability of parked state.

3.4 V3–V4: calibration

At concurrency $k \in \{1, 2, 4, 8, 16, 32\}$, per-sequence output rate fits

$$r(k) = \frac{R}{k + k_{1/2}} \quad (2)$$

with $R = 156.19$, $k_{1/2} = 4.606$, and $R^2 = 0.99995$. The original assumed $k_{1/2} = 16$, so its batching model was materially too optimistic. Three clean server process starts with a warm host page cache take 38.163, 38.141, and 38.161 s (mean 38.155 s). This is not a disk-cold or 70B cold start.

3.5 P1: survival under delay and pressure

We park a 16k-token target, immediately issue 0, 4, 8, 16, or 24 distinct 4k-token neighbors, and then wait if necessary so the total park interval reaches the minimum target $\tau_{\min} \in \{0, 8, 32\}$ s before probing. We normalize probe TTFT between the same-trial hit and miss to estimate retained prefix fraction. All 45 trials completed. At $N \leq 8$, retention is essentially one; at $N = 16$, it is 0.645; and at $N = 24$, it is approximately zero. The pressure work itself takes about 12.2 s and 18.2 s at $N = 16$ and $N = 24$, respectively, so it overruns the 0/8-s minimum targets. Across the distinct elapsed times available at each pressure level, allocation pressure rather than wall-clock age determines survival in this range.

This result also exposes partial eviction. The measured capacity is 75,216 tokens; a 16k target plus sixteen 4k neighbors exceeds it by about 4.8k tokens, consistent with a partial rather than all-or-nothing miss.

3.6 P2: pressure-aware admission on hardware

We repeat the 24-neighbor case in two modes. Uncontrolled submits all 24 before probing the target. Protected submits eight, probes, and only then drains the remaining sixteen. Every neighbor completes in both modes. Across three trials, uncontrolled survival is zero and mean probe TTFT is 3.735 s; protected survival is one and TTFT is 0.124 s. The test is not a cluster autoscaler, but it isolates the admission mechanism: delaying excess pressure preserves the reclaimable prefix without rejecting work.

4 Corrected Method

4.1 Real and constructed inputs

Arrival time, context length, and generated length come directly from the Azure traces. The traces contain neither program identifiers nor tool durations. We construct programs of $T \in \{1, 4, 16, 64\}$ turns and sweep mean tool delay $\tau \in \{0, 8, 30\}$ s. $T = 1, \tau = 0$ is the unmodified request-level control. Context accumulates across constructed turns, but every prompt plus decode is capped at the validated 32,768-token server limit. Later appended content is clipped at that boundary, so the $T = 64$ sensitivity never asks the simulator to serve a request rejected by the measured server. We use seed 20260720.

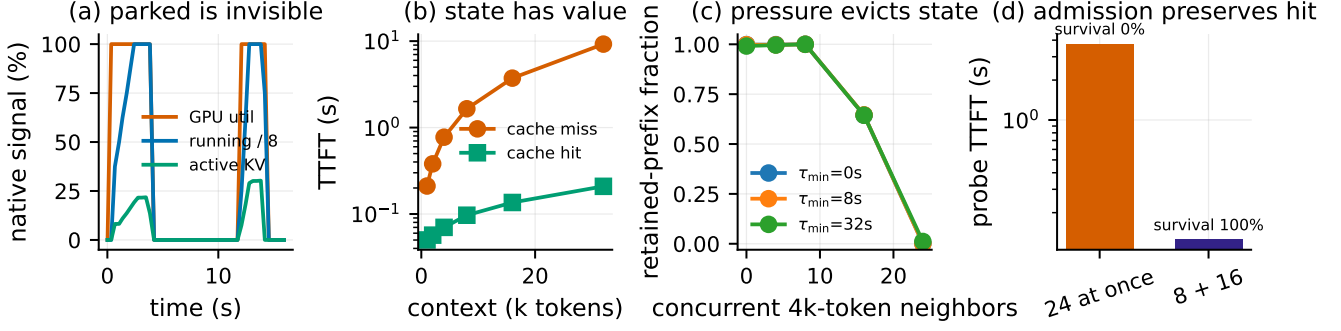


Figure 2: Hardware validation. (a) During tool-call intervals, all three native signals drop to zero. (b) Cache loss has a context-proportional TTFT cost. (c) A parked prefix survives elapsed time but not sufficient concurrent pressure. (d) P2 defers work beyond the P1 limit and preserves the hit.

4.2 State model and service calibration

Each replica separately tracks active KV allocation and reclaimable cached content. A completed turn releases its active allocation but leaves its prompt and generated tokens as LRU content. Tool output is new input and enters KV only when the next request prefills it; it is never counted as recomputation. New active blocks reclaim the oldest parked prefixes at token granularity. Scale-in removes only idle replicas and discards cached content on them. On return, the program prefills new tool tokens plus any missing part of its old prefix.

Defaults use the measured 75,216-token capacity, $k_{1/2} = 4.606$, and 38.155 s process cold start. Jointly fitting V1 and V3 gives a service rate of 19,607 token-equivalents/s and decode weight 94.282 relative to prefill. Maximum batch is 32. E7 explicitly sweeps capacity, batch, and policy target.

We simulate 2,400 s of arrivals, exclude a 300 s warm-up from scoring, then stop arrivals and drain for the larger of 600 s or a workload-derived bound. This drain is essential: scoring only to the last arrival mechanically labels late long programs as failures. Unfinished work after the drain is an SLO failure and program goodput is reported separately; pressure admission has goodput one at all eight E8 points, while some baselines do not.

The SLO is per-turn slowdown at most three. For each workload we search for the smallest static cluster achieving at least 95% SLO attainment. Dynamic cost is reported as GPU-seconds relative to that reference. Unfinished admitted work, if any, is counted as an SLO failure.

4.3 Policies

We implement HPA on GPU utilisation (70% target and 300 s scale-down stabilisation), KEDA on waiting requests, KV-util on active KV (80% target), a Holt arrival forecaster, and tabular Q-learning with and without parked count in its

state [8, 9]. The candidate PARKAWARE policy uses

$$N = \max\left(\left\lceil \frac{\text{running} + \text{queued} + \text{parked}}{B\theta} \right\rceil, \left\lceil \frac{\text{active KV}}{K\theta} \right\rceil\right). \quad (3)$$

It is retained unchanged as the direct test of the original idea. Under the corrected mechanism, its parked term is a conservative seat reservation, not a measurement of pinned memory.

The new pressure-aware baseline instead admits at most eight active turns per replica, rejects any placement that would reclaim a parked prefix, and scales out on the larger of queued compute demand and total resident-cache pressure. The threshold of eight comes directly from P1. This is a deliberately simple admission baseline, not an oracle: it does not estimate reuse probability, prefix value, or optimal packing.

5 Results

5.1 Parked work is common and natively invisible

On a fixed eight-replica cluster, the simulated parked fraction is 58.7% at $(T, \tau) = (4, 4)$, 88.4% at $(8, 8)$, and 91.0% at $(16, 8)$. At those points, respectively 13.3%, 23.8%, and 19.5% of post-warm-up samples contain parked programs while running, GPU, and active-KV signals are all zero. Figure 3 distinguishes how much work is parked from how often its invisibility is complete.

5.2 Counting parked programs is not a robust policy

Table 1 reverses the uploaded paper’s headline. PARKAWARE loses to HPA and pressure admission in mean SLO, and no dynamic policy simultaneously matches the cheapest static SLO configuration and costs less on average. Pressure admission has the best mean SLO but the lowest SLO/cost ratio because strict protection holds $2.36\times$ the static GPU

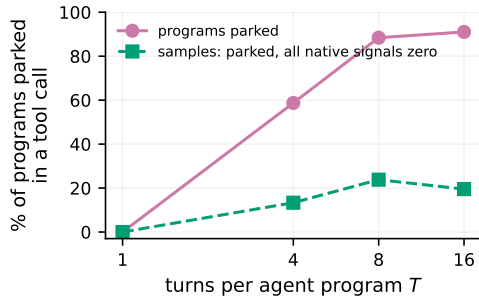


Figure 3: Simulated parked fraction and completely invisible parked intervals on a fixed cluster. Reclaimable cache is tracked internally but is not a native autoscaling signal.

Table 1: Mean over the six coding-trace agentic points ($T > 1, \tau > 0$).

Policy	SLO	GPU / static	SLO / cost
Pressure admission	0.986	2.357	0.418
HPA (GPU)	0.936	2.105	0.445
KEDA (queue)	0.552	0.993	0.556
KV-util	0.874	1.206	0.725
Predictive	0.678	0.806	0.841
RL (Q-learning)	0.791	1.078	0.734
RL + parked	0.774	0.998	0.775
PARKAWARE candidate	0.902	1.220	0.739

budget. Predictive has the best ratio precisely because it tolerates substantial SLO loss. Queue-inclusive slowdown therefore turns the result into an explicit reliability–cost tradeoff, not a universal winner.

The result varies by regime. At $T = 4, \tau = 8$, HPA attains 1.000, PARKAWARE 0.702, and pressure admission 0.935. At $T = 16, \tau = 30$, KV-util leads at 0.995, followed by HPA at 0.993, pressure admission at 0.991, and PARKAWARE at 0.980. At $T = 64, \tau = 8$, HPA, PARKAWARE, and pressure admission attain 1.000, 1.000, and 0.997, while KV-util is 0.906. Such crossovers are exactly what one proxy should produce when prefix value depends on context size, return timing, cache pressure, and service batching.

5.3 Pressure dominates scale-in damage

At $T = 8, \tau = 8$, HPA recomputes 1.71M tokens, of which only 0.245M are attributed to scale-in. KEDA, KV-util, and PARKAWARE recompute 8.42M, 6.54M, and 9.46M tokens. Their scale-in components are only 0.491M, 0.226M, and 0.311M. Aggregated across these four baseline policies at $T = 2, 4, 8, 16$, pressure eviction accounts for 96.53% of destroyed prefix tokens and scale-in for 3.47%. The pressure share is 95.71%, 93.30%, 95.13%, and 98.00% at the four turn counts, so it is not a single-point artifact. This agrees with P1’s independent hardware evidence: elapsed time barely moves survival while enough neighboring contexts erase it.

Pressure-aware admission eliminates pressure eviction at every E4 point. It does not eliminate scale-in loss, and strict protection costs replicas: at $T = 8$ it attains 0.975 SLO with 15.53k GPU-seconds versus HPA’s 0.997 with 15.95k and KV-util’s 0.855 with 6.71k. The old equation that charged every parked context only when a replica was removed therefore described the wrong dominant mechanism, while a controller that protects every surviving prefix can also overpay.

5.4 Cold start and workload change the ordering

At the measured 38.155 s process cold start, HPA, KEDA, Predictive, and PARKAWARE attain 0.997, 0.743, 0.472, and 0.720 at $(T, \tau) = (8, 8)$. At 180 s they attain 0.567, 0.680, 0.371, and 0.471. At 300 s KEDA attains 0.613, exceeding HPA’s 0.423 and PARKAWARE’s 0.349 because its conservative queue response avoids some extremely expensive reconstruction. The original monotone ParkAware advantage does not survive correction. Pressure admission ranges from 0.928 to 1.000; at 300 s it attains 0.948 but uses 17.0k GPU-seconds. SLO protection is not free.

On the conversation trace constructed as $T = 8, \tau = 8$, HPA attains 1.000, KV-util 0.802, and PARKAWARE 0.965; KEDA attains 0.174. On the coding trace at the same point, the values are 0.997, 0.855, 0.720, and 0.743, respectively. Pressure admission attains 1.000 on both. A program-level policy cannot be evaluated independently of the token and arrival distribution beneath the constructed program structure.

5.5 Sensitivity at realistic trajectory length

The original E7 used $T = 8$. We rerun it at $T = 64$, closer to long agent trajectories while enforcing the 32k sequence limit. At measured capacity and batch 32, HPA and PARKAWARE attain 1.000, pressure admission 0.997, and KV-util 0.906. Across the $T = 64$ capacity sweep HPA remains 1.000, while pressure admission ranges from 0.851 at 31k to 1.000 at 150k. Changing PARKAWARE’s target from 0.5 to 0.9 moves attainment from 1.000 to 0.906. Capacity, batching, and control target remain material even after trajectory length is fixed.

5.6 High load exposes a useful pressure-control regime

The base trace is light. E8 deterministically superposes phase-shifted copies of the coding trace at 1, 2, 4, and $8\times$ load. This is a synthetic stress test, not a measured production rate. At $T = 8$, the static reference grows from 3 to 5, 8, and 14 replicas; at $T = 64$, it grows from 3 to 6, 12, and 24. Every point includes Static, HPA, KEDA, KV-util, Holt predictive, Q-learning with and without parked state, PARKAWARE, and pressure admission. RL is trained on a disjoint trace prefix for each turn/load point.

At $T = 8$ and $4\times/8\times$, pressure admission attains 0.999/1.000 SLO with 1.389/1.461 $\times$ static GPU cost. HPA

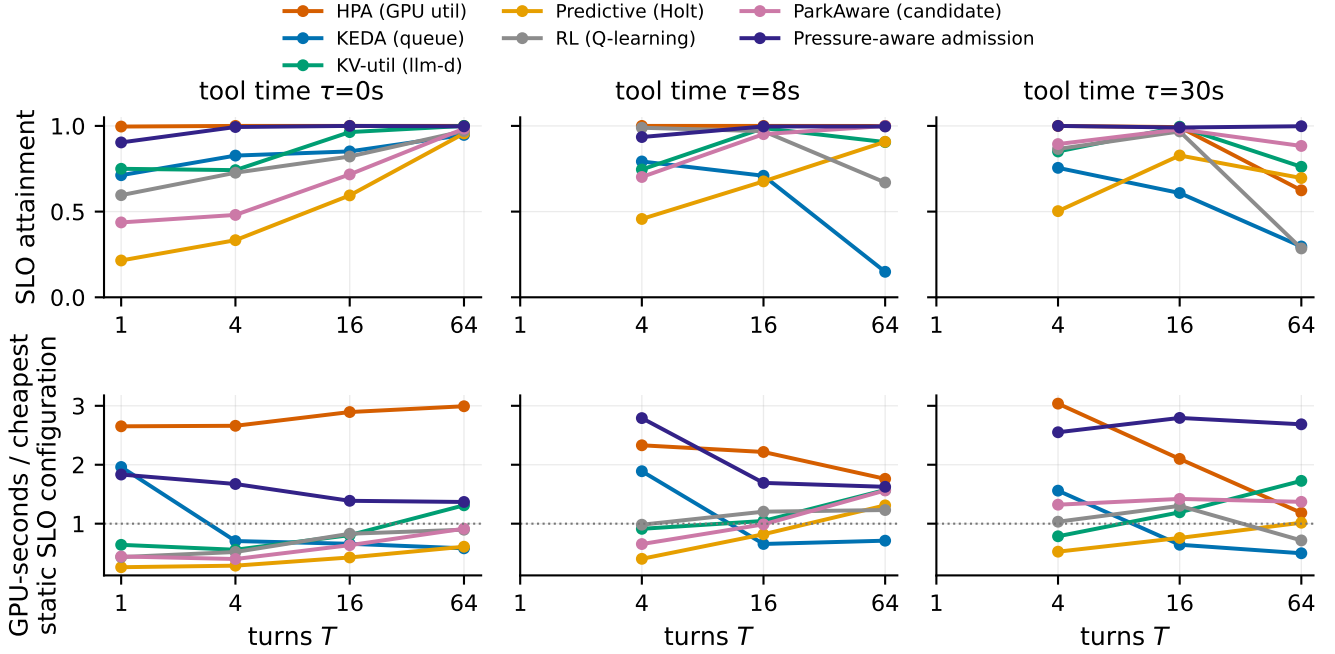


Figure 4: SLO attainment and GPU cost relative to the cheapest static SLO-meeting cluster. Pressure-aware admission protects SLO at most points but pays for that protection; no dynamic policy dominates on both axes.

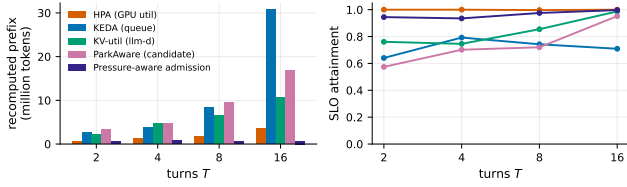


Figure 5: Recomputed prefix tokens and SLO on the coding trace. Recompute includes partial pressure eviction and cache loss on scale-in.

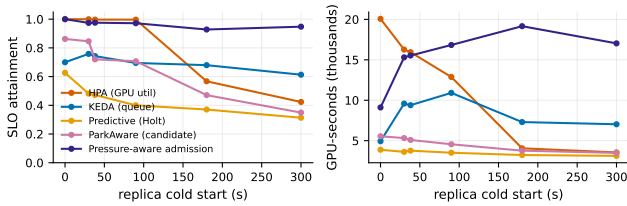


Figure 6: Cold-start sensitivity at $T = 8, \tau = 8$. The measured 14B point is 38.155 s; larger values are sensitivity cases, not measurements.

attains 1.000/0.997 with 2.743/2.687 $\times$, so protection uses 49.4%/45.6% fewer GPU-seconds. At $T = 64$, the reliability benefit persists but the GPU result reverses. At 8 $\times$, pressure

admission attains 0.991 versus HPA’s 0.979 and recomputes 326M fewer tokens, but costs 1.335 versus 1.234 $\times$ static, 8.2% more GPU. All pressure-admission points have program goodput one. This is a useful pressure-control regime, not a universal cost optimum.

5.7 The P1 threshold is robust, not universally optimal

E9 sweeps seven per-replica admission batches from 4 through 16 at 4 $\times$ /8 $\times$ load and both turn counts. The hardware-derived value eight attains 0.999 and 1.000 at $T = 8$, and 0.998 and 0.991 at $T = 64$; all four have goodput one and zero pressure evictions. The exact optimum varies: at $T = 64, 8\times$, batch four attains 0.999 versus batch eight’s 0.991, whereas at $T = 8, 8\times$, batches 6–10 all attain 1.000. The capacity-only neighbor bound is 14, but P1 observed intact prefixes only through eight. Eight is therefore a measured safe operating point, not a tuned universal optimum.

5.8 Admission exchanges queueing for avoided recomputation

E10 compares each E8 policy with HPA at the same point. At $T = 8$, pressure admission adds 3.37k and 5.63k aggregate queue-seconds at 1–2 $\times$, and its SLO is 1.8–2.1 points lower. At 4–8 $\times$, that queue delta falls to 70.5 and zero seconds while it saves 3.93M and 16.65M recomputed tokens

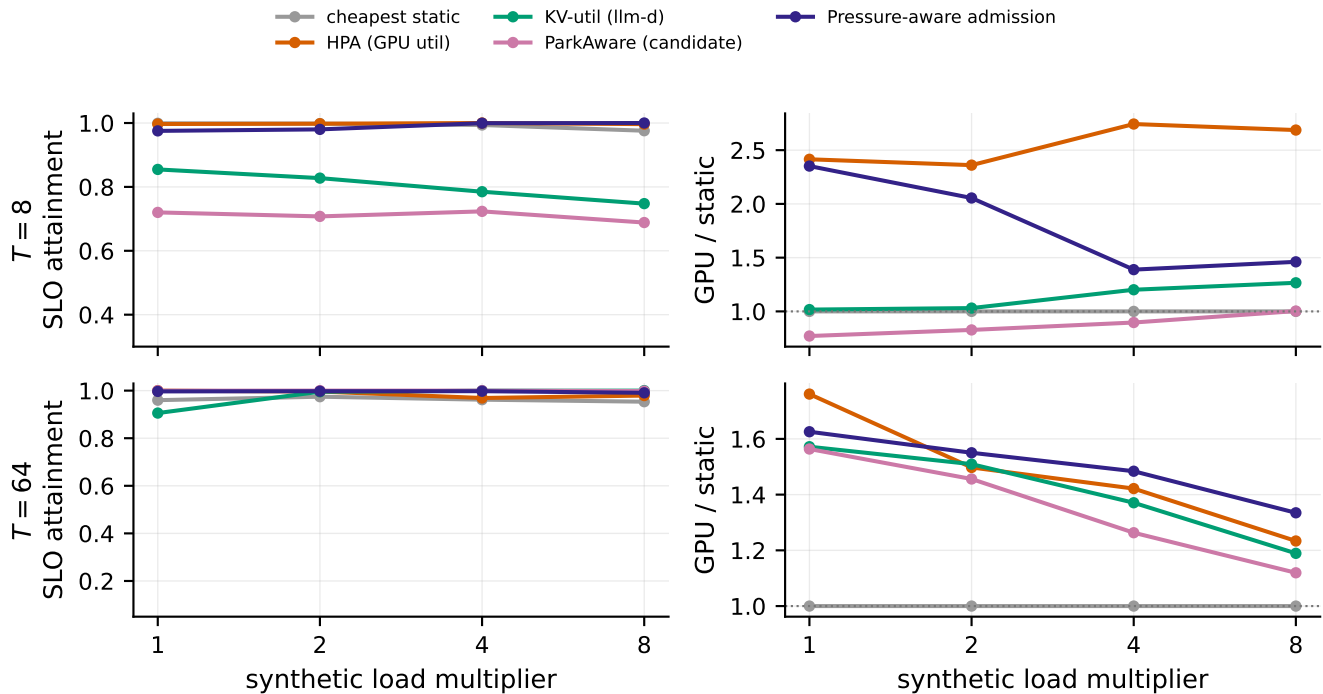


Figure 7: E8 queue-inclusive load stress. Pressure admission is both reliable and cheaper than HPA at high load for $T = 8$; at $T = 64$ it trades modestly more GPU for less recomputation and higher SLO.

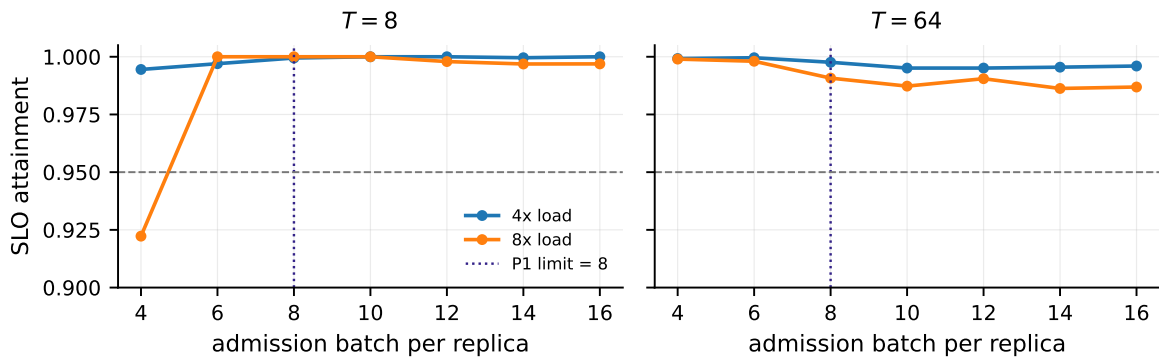


Figure 8: E9 admission sensitivity. The measured threshold is robust across the four stress points, while more conservative admission helps the longest, highest-load trajectory.

and 45.6–49.4% GPU. At $T = 64$, it adds 0.28–4.55k queue-seconds and saves 41–326M recomputed tokens. The saved work improves SLO by 1.2–2.9 points at 4–8 $\times$, but total GPU rises 4.4–8.2% because preserving many long prefixes requires more replicas. Queueing is bounded and visible; eviction creates large, workload-dependent hidden work.

6 Implications

Expose cache state, not only allocation. Active-KV utilisation correctly reports allocated blocks, but an autoscaler

also needs the amount, age, ownership, and eviction priority of reclaimable prefix content. Conflating the two would make admission control unsafe; exporting both would preserve the distinction.

Value is probabilistic. A parked prefix is worth approximately its miss penalty times the probability that the program returns before pressure evicts it. Tool type, transcript size, recent reuse, and cache competition may predict that value. A raw parked count assigns the same value to a 1k

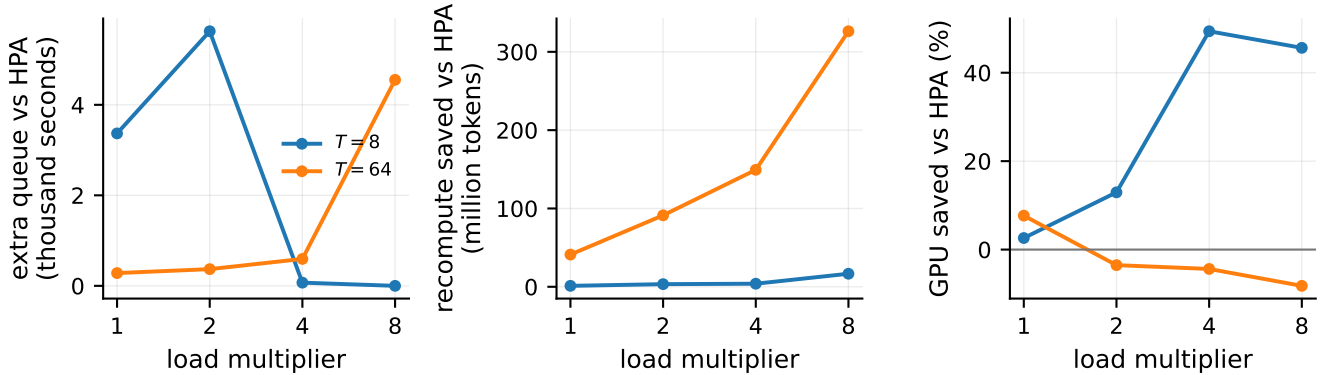


Figure 9: E10 mechanism accounting relative to HPA. Avoided recomputation grows with trajectory length, but GPU savings do not: at $T = 64$ reliability costs additional replicas under high load.

prefix and a 32k prefix despite a measured 56-fold difference in recompute penalty.

Admission is the first pressure-control lever. Scale-in is destructive when it targets a replica with valuable surviving prefixes, but it explains only 3.47% of baseline token loss here. Admission and placement act before the dominant event: a neighbor reclaiming those blocks. A practical controller should reject prefix-destructive placement, scale out when the rejected work is worth serving, and still allow low-value prefixes to be reclaimed. Our strict baseline implements the first two pieces and exposes the cost of omitting the third.

7 Threats to Validity

Single hardware configuration. Four modelling assumptions and the P1 and P2 pressure mechanisms are measured on a single vLLM instance. One assumption was falsified by that measurement and the model corrected accordingly. Results may differ across engines, versions, models, cache managers, and hardware. V4 measures a process restart with warm host page cache, not a machine reboot or disk-cold start.

Constructed program structure. Azure supplies request records, not agent identities or tool intervals. Our T and τ are constructed. We cap the constructed context at the measured 32k sequence limit, which represents bounded context but not a specific production summarization algorithm. Sweeps and a $T = 1$ control expose, but do not eliminate, this limitation.

Simulator rather than cluster replay. Hardware measurements calibrate the mechanism and service curve; E2–E10 are still simulations. The model uses LRU pressure, one model per replica, and simplified placement. Production schedulers may migrate, preempt, offload, or explicitly pin cache state.

Synthetic high load. E8 superposes phase-shifted copies of one trace. It preserves within-copy request sizes and burst

structure but is not an independent observation of a busier deployment. Its role is to test whether the low-load conclusion persists once static demand reaches 8–24 replicas.

Policy implementations. The policies follow their documented signal and standard control forms, but deployment-specific tuning can change results. The tabular RL baseline is deliberately conventional, not a claim about all learning-based controllers. We present PARKAWARE as a falsified candidate and pressure admission as a data-directed baseline, not production recommendations.

8 Related Work

PagedAttention and vLLM make KV management efficient within a serving instance [3]. Agent-serving systems increasingly schedule whole programs rather than isolated requests; Autellix, for example, uses cumulative program service time [4]. This work addresses the interface between that program state and cluster-level elasticity.

LLM autoscaling work optimizes token-aware or disaggregated serving. TokenScale scales on token velocity, while ServerlessLLM attacks model-loading cold start directly [5, 6]. DistServe and DynamoLLM optimize cluster organization and goodput [11, 12]. Our measurement shows that active request metrics omit a distinct object: reclaimable, application-owned prefix value between requests.

Classical elasticity separates reactive, proactive, and learning-based controllers [7, 9]. RL resource allocation has a long history [8]. Our corrected result is compatible with that literature’s main lesson: policy quality depends on the state representation. Here, neither native active-resource signals nor a parked-program count represents prefix survival and value.

9 Conclusion

A parked agent does not hard-hold KV in native vLLM, but it may retain a cheap path back into execution. That path

has measurable value, is invisible to native autoscaling metrics, and disappears under neighboring cache pressure. Correcting this distinction invalidates both our original mechanism and its claimed ParkAware advantage. Hardware and simulation then identify the same enemy: not elapsed parking time and rarely scale-in, but pressure from admitted work. Hardware admission preserves the hit without dropping work, and a simple pressure-aware baseline eliminates simulated pressure eviction. It is both more reliable and cheaper than HPA for short trajectories under high load; for long trajectories it is more reliable but can cost more GPU. Agentic autoscaling needs explicit reclaimable-prefix state, reuse value, and pressure-aware admission—not another idle-time scalar.

A Sensitivity Figure

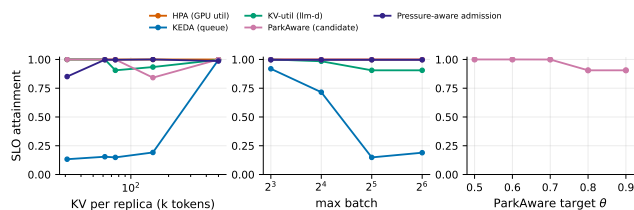


Figure 10: Sensitivity rerun at $T = 64, \tau = 8$: KV capacity, maximum batch, and the candidate ParkAware target.

References

- [1] P. Patel et al. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024. Traces: <https://github.com/Azure/AzurePublicDataset>.
- [2] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [3] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.
- [4] M. Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. arXiv:2502.13965, 2025.
- [5] R. Lai et al. TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity. arXiv:2512.03416, 2025.
- [6] Y. Fu et al. ServerlessLLM: Low-latency serverless inference for large language models. In *OSDI*, 2024.
- [7] N. Herbst, S. Kounev, R. Reussner. Elasticity in cloud computing: What it is, and what it is not. In *ICAC*, 2013.
- [8] E. Barrett, E. Howley, J. Duggan. Applying reinforcement learning towards automating resource allocation and application scalability in the cloud. *Concurrency and Computation*, 25(12), 2013.
- [9] Y. Garí et al. Reinforcement learning-based application autoscaling in the cloud: A survey. *Engineering Applications of Artificial Intelligence*, 2021.
- [10] Polar: Agentic RL on any harness at scale. arXiv:2605.24220, 2026.
- [11] J. Stojkovic et al. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *HPCA*, 2025.

- [12] Y. Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. In *OSDI*, 2024.